

ReCA: Integrated Acceleration for Real-Time and Efficient Cooperative Embodied Autonomous Agents

Zishen Wan
Georgia Institute of Technology
Atlanta, GA, USA
zishenwan@gatech.edu

Yuhang Du
University of Minnesota, Twin Cities
Minneapolis, MN, USA
voyager.yuhang.du@gmail.com

Mohamed Ibrahim
Georgia Institute of Technology
Atlanta, GA, USA
mibrahim81@gatech.edu

Jiayi Qian
Georgia Institute of Technology
Atlanta, GA, USA
jiayiqian@gatech.edu

Jason Jabbour
Harvard University
Cambridge, MA, USA
jasonjabbour@g.harvard.edu

Yang (Katie) Zhao
University of Minnesota, Twin Cities
Minneapolis, MN, USA
zhao1948@umn.edu

Tushar Krishna
Georgia Institute of Technology
Atlanta, GA, USA
tushar@ece.gatech.edu

Arijit Raychowdhury
Georgia Institute of Technology
Atlanta, GA, USA
arijit.raychowdhury@ece.gatech.edu

Vijay Janapa Reddi
Harvard University
Cambridge, MA, USA
vj@eecs.harvard.edu

Abstract

Cooperative embodied systems, where multiple agents collaborate through integrated perception, planning, action, and advanced reasoning powered by large language models (LLMs), show great potential for tackling complex, long-horizon, multi-objective tasks in real-world environments. Despite these algorithmic advancements, deploying embodied agents on current systems remains challenging due to prolonged planning and communication latency, limited scalability, and heightened sensitivity in low-level execution, all of which lead to significant system inefficiencies.

This work proposes ReCA, a characterization and co-design framework dedicated to cooperative embodied agent system acceleration, aiming to enhance both task efficiency and system scalability. On the algorithm level, ReCA enables *efficient local model processing* to alleviate the substantial model costs. On the system level, ReCA presents a *dual-memory structure* with integrated long-term and short-term memory, *hierarchical cooperative planning scheme* with centralized and decentralized cooperation, and *planning-guided multi-step execution* for highly efficient and scalable cooperative embodied agent computation. On the hardware level, ReCA employs a *heterogeneous hardware system* with high-level planning GPU subsystem and low-level planning accelerator subsystem to ensure efficient and robust task execution. Evaluated across long-horizon multi-objective tasks, ReCA generalizes across application scenarios and system scales,

achieving a 4.3% increase in successful missions with 10.2× speedup compared to the state-of-the-art cooperative embodied autonomous agent systems.

CCS Concepts: • Computer systems organization → Embedded and cyber-physical systems; Architectures.

Keywords: Autonomous Agents, Embodied Systems, Cooperative Intelligence, Hardware Accelerator

ACM Reference Format:

Zishen Wan, Yuhang Du, Mohamed Ibrahim, Jiayi Qian, Jason Jabbour, Yang (Katie) Zhao, Tushar Krishna, Arijit Raychowdhury, and Vijay Janapa Reddi. 2025. ReCA: Integrated Acceleration for Real-Time and Efficient Cooperative Embodied Autonomous Agents. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*, March 30–April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3676641.3716016>

1 Introduction

Embodied AI systems consist of agents that interact with the physical world, integrating perception, cognition, and action to accomplish complex tasks [13, 47, 48, 76, 81, 84]. The rapid advancement of embodied AI is driving transformative changes in robotics and autonomous systems, enabling real-world applications across domains. These systems typically rely on perception, memory, communication, planning, and action modules, as a coherent workflow for task execution.

A recent advancement of embodied AI is **cooperative embodied AI systems**, where multiple autonomous agents collaborate to achieve shared goals [11, 12, 20, 21, 26, 29, 30, 43, 75, 83, 86–88, 90]. These systems typically employ modular frameworks to perform long-horizon tasks, integrating high-level planning and reasoning with low-level action execution. LLMs are increasingly utilized in these



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/2025/03.

<https://doi.org/10.1145/3676641.3716016>

systems to enhance communication, decision-making, and planning in dynamic, complex environments. Cooperative embodied agent systems have demonstrated superior capabilities in long-horizon multi-objective tasks. For instance, CoELA [86] improves task success rate by 23% in benchmarks involving complex object transportation, table setting, and dishwashing in human-agent interaction scenarios.

Cooperative embodied agents have demonstrated impressive long-horizon task integration and planning capabilities using distributed GPU clusters. However, enabling *real-time and efficient* cooperative embodied AI systems—highly desirable for many human-AI applications—remains a challenging open problem. For instance, we observe that current cooperative embodied systems, such as CoELA [86], COMBO [87], and MindAgent [20], are highly inefficient, requiring 18, 23, and 21 minutes, respectively, even on desktop GPUs, to complete a single long-horizon multi-objective task.

To understand this further, we systematically profile and analyze the runtime, scalability, and module sensitivity of various cooperative embodied AI systems and identify the following system-level challenges. **① Large Planning and Communication Latency.** Cooperative embodied agents typically rely on LLM-based modules for high-level planning and inter-agent communication. Each action step involves multiple sequential LLM executions, making real-time operation impractical. **② Limited Scalability.** As the number of embodied agents increases in complex, long-horizon tasks, decentralized systems experience exponential latency growth, while centralized systems suffer from a sharp decline in task success rates due to reduced cooperation efficiency. **③ Sensitivity of Low-Level Execution.** Typically, the embodied agents transform LLM-generated high-level plans to low-level planning and control commands for task execution, making the efficient and robust low-level execution critical for mission success.

To address the aforementioned challenges, we develop ReCA, an integrated acceleration framework designed to enhance the real-time efficiency and scalability of cooperative embodied autonomous agent systems. **On the algorithm level,** ReCA presents *efficient local model processing* to eliminate API dependencies by deploying LLMs locally on agents while maintaining comparable performance in cooperative long-horizon tasks. **On the system level,** ReCA introduces a *dual-memory structure* with hybrid long-term and short-term memory to improve planning efficiency, *hierarchical cooperative planning scheme* with intra-cluster centralized and inter-cluster decentralized cooperation to improve system scalability, and *planning-guided multi-step execution strategy* to mitigate sequential critical-path dependencies. **On the hardware level,** ReCA employs a *heterogeneous hardware system* with high-level planning GPU subsystem and low-level planning A-star accelerator subsystem to ensure efficient and robust task execution. Evaluated across three cooperative embodied systems and six benchmarks, ReCA achieves an

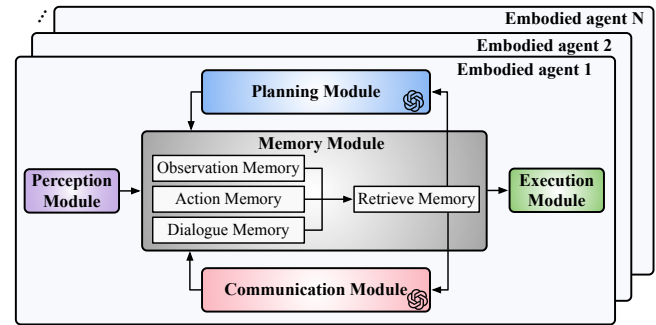


Figure 1. Modular framework for cooperative embodied AI agents. The agentic modular framework comprises five core modules: the *perception module* interprets the environment, the *communication module* formulates messages, the *planning module* devises high-level plans, the *execution module* translates plans into primitive actions, and the *memory module* retains the agent’s knowledge and experiences across observation, action, and dialogue memories.

average of 10.2× end-to-end speedup, demonstrating its effectiveness in accelerating cooperative AI-driven autonomy.

This paper, therefore, makes the following contributions:

- We systematically analyze the runtime, scalability, and modular sensitivity of cooperative embodied AI agent systems, identifying the primary causes of inefficiency and potential optimization opportunities (Sec. 3).
- We introduce ReCA, an integrated acceleration framework designed to enable real-time, efficient, and scalable cooperative embodied AI systems (Sec. 4). Our innovations span both system-level optimizations (Sec. 5) and hardware acceleration (Sec. 6), enhancing cooperative efficiency and reducing end-to-end task latency.
- We evaluate ReCA on long-horizon multi-objective tasks and cooperative embodied agent systems, demonstrating it generalizes across scenarios and scales, achieving a 4.3% increase in successful missions with a 10.2× speedup over state-of-the-art embodied systems (Sec. 7).

2 Cooperative Embodied Agent Workflow

This section first presents the modular framework for cooperative embodied AI agents (Sec. 2.1), and then introduces three representative cooperative embodied agent systems for detailed analysis (Sec. 2.2).

2.1 Modular Framework

Embodied agents typically harnesses the strong reasoning, planning and communication capabilities of LLMs to cooperatively tackle complex, long-horizon, multi-objective tasks. We present a modular framework for cooperative embodied agent systems, comprising five key modules: perception, memory, communication, planning, and execution (Fig. 1). This framework encompasses a wide range of embodied agent architectures and interaction mechanisms [20, 30, 43, 49, 59, 86, 87]. The details of each module are as follows:

Table 1. Cooperative embodied agent systems. A selection of systems analyzed, representing a diverse range of cooperative scenarios.

Embodied Systems	Modular Framework Details					Tasks	Benchmarks
	Perception	Memory	Communication	Planning	Execution		
CoELA [86]	Mask R-CNN	Observation, dialogue, action	GPT-4	GPT-4	A-star planner	Collaborative objects transporting	TDW-MAT [15]
COMBO [87]	Diffusion model	Observation, dialogue, action	LLaVA-1.5	LLaVA-1.5 + Tree search	A-star planner	Collaborative housework (e.g., set up table)	C-WAH [63]
						Collaborative gaming	TDW-Game [14]
						Collaborative housework (e.g., cook)	TDW-Cook [14]
MindAgent [20]	N/A	Observation, dialogue, action	GPT-4	GPT-4	A-star planner	Collaborative gaming	CuisineWorld [20]
						Collaborative exploration and navigation	Minecraft [22]

Perception module. This module processes sensor data, enabling agents to collect and analyze critical information for higher-level reasoning. It constructs a global or shared environmental model, incorporating spatial layouts, dynamic entities, obstacles, and resource locations. Each agent continuously updates its local environmental view through sensor inputs and communication with other agents.

Memory module. Embodied agents retain knowledge and experiences from interactions with their environment and peers across three categories: observation, dialogue, and action memory. *Observation Memory* stores the agent’s understanding of the world, including maps, task progress, and agent statuses, continuously updated by the Perception Module. *Dialogue Memory* logs communication history and past interactions. *Action Memory* tracks the agent’s actions, execution status, and accumulated knowledge on implementing high-level plans across different environments, encoded as code or neural model parameters.

Communication module. This module retrieves relevant data from memory, including environmental maps, task progress, agent states, and past interactions. It converts this data into text-based descriptions, which are then used to prompt LLMs to generate meaningful messages for communication with other agents.

Planning module. The planning module extracts relevant information from memory and generates high-level plans based on the current state and stored procedural knowledge. LLMs use this data to formulate executable plans without relying on few-shot demonstrations. To enhance reasoning, zero-shot chain-of-thought prompting [34] can be applied, guiding LLMs through a more structured deliberation process before finalizing decisions.

Execution module. While LLMs are effective at high-level planning, they struggle with low-level planning and control [79, 89]. To bridge this gap, the execution module translates high-level plans into primitive actions for robust, low-level execution [18]. This design ensures generalizability, allowing the system to adapt to diverse tasks while leveraging LLMs’ world knowledge and reasoning abilities.

2.2 Representative Workloads

Building on the modular framework (Fig. 1) and [71], we select and analyze three representative cooperative embodied systems in detail: CoELA [86] for cooperative object transportation and household tasks, COMBO [87] for cooperative gaming and cooking, and MindAgent [20] for cooperative

Task Goal: Put one pear, two apples and one pudding onto the plate.



Figure 2. Example of a cooperative embodied task. A complex multi-agent collaboration scenario that integrates sensory inputs, communication, and long-horizon, multi-objective task execution.

human-AI planning. These systems achieve state-of-the-art performance, demonstrating advanced long-horizon planning and decision-making capabilities. Our goal is to identify system-level challenges and facilitate scalable deployment of embodied AI, where latency and efficiency are critical.

CoELA. *Cooperative Embodied Language Agent (CoELA)* is designed for decentralized environments, enabling embodied agents to collaborate with each other or with humans on long-horizon, multi-objective tasks. Fig. 2 illustrates a cooperative scenario involving two agents. CoELA integrates Mask R-CNN for perception and GPT-4 for planning and communication, demonstrating strong performance on the TDW-MAT [15] and C-WAH [63] benchmarks. It excels in perceiving complex environments, reasoning about the world and other agents, communicating efficiently, and executing long-horizon plans in tasks such as collaborative object transport, tea preparation, table setting, and dishwashing.

COMBO. *Compositional Model for Embodied Multi-Agent Cooperation (COMBO)* is a decentralized planning framework leveraging compositional world models for online cooperative planning. Agents reconstruct the global world state from partial egocentric observations using a diffusion model [33] and utilize Visual Language Models (LLaVA-1.5) [42] to infer other agents’ intents, communicate, and propose actions. COMBO further refines action sequences through tree search, achieving strong performance in long-horizon cooperation tasks such as cooking and puzzle-solving in ThreeDWorld [14] (TDW-Game and TDW-Cook). It reduces the number of steps required by 45% compared to LLaVA [42].

MindAgent. *MindAgent* MindAgent is designed for multi-agent collaboration in gaming and household tasks, enabling agents to coordinate and execute complex plans in a centralized manner with emergent planning capabilities. It utilizes LLMs (GPT-4) for task scheduling and cooperation, enhancing planning efficiency through few-shot prompting

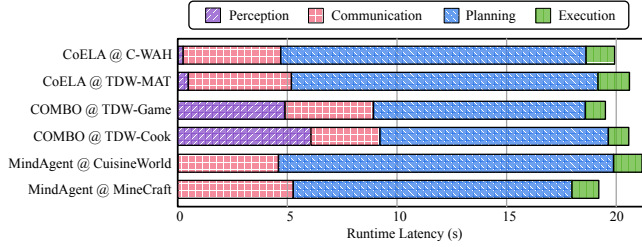


Figure 3. Performance bottleneck analysis. Latency distribution across module in CoELA [86], COMBO [87], and MindAgent [20] workloads, measured across six benchmarks.

and feedback mechanisms. Evaluations in CuisineWorld [20] and Minecraft [22] demonstrate significant improvements in multi-agent coordination and collaboration efficiency.

3 System Characterization and Implication

This section examines the system behavior of various cooperative embodied agent systems (Sec. 3.1, Sec. 3.2), evaluates their scalability (Sec. 3.3) and module sensitivity (Sec. 3.4), to provide insights into their performance characteristics.

3.1 Profiling Setup

To assess the efficiency of cooperative embodied AI systems, we profile CoELA [86], COMBO [87], and MindAgent [20] in terms of runtime latency, scalability, and module sensitivity, focusing on their ability to handle long-horizon, multi-objective tasks [71]. Each workload follows the configurations reported in its respective study.

CoELA workload setup. We profile CoELA on the C-WAH [63] and TDW-MAT [15] benchmarks, where two agents perform various household tasks. The objective is to complete 3 to 5 subgoals per task within 250 time steps. Efficiency is measured using the *Average Steps* metric, which quantifies the average steps taken per task. CoELA utilizes GPT-4 via the OpenAI API for communication and planning, while actions are executed on Intel i7 CPU.

COMBO workload setup. We evaluate COMBO on the TDW-Cook [14] and TDW-Game [14] benchmarks, which involve complex embodied multi-agent planning. In these tasks, embodied agents cooperate to explore, navigate, and place food items to create dishes in the indoor environments. LLaVA-1.5 model inferences are executed on NVIDIA A6000 GPU, while agent actions run on Intel i7 CPU.

MindAgent workload setup. We profile MindAgent with GPT-4 on the CuisineWorld [20] and Minecraft [22]. These benchmarks represent gaming and household scenarios that assess multi-agent collaboration efficiency. The system is tasked with managing multiple agents to complete as many missions as possible while optimizing task execution.

3.2 Performance Bottleneck Analysis

End-to-end latency. Fig. 3 presents the end-to-end latency per step for the CoELA, COMBO, and MindAgent in cooperative long-horizon, multi-objective tasks. The analysis reveals

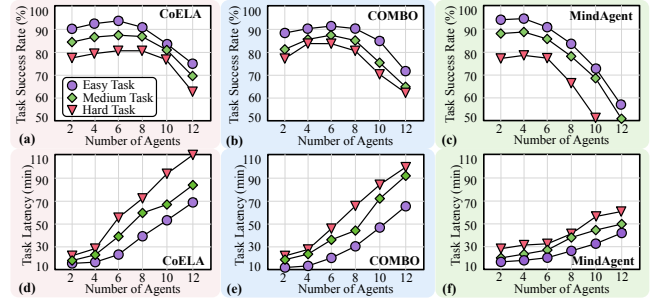


Figure 4. System scalability analysis. Average task success rate and end-to-end latency for CoELA [86], COMBO [87], and MindAgent [20] systems across varying number of agents and tasks.

three key findings: (1) *High system runtime latency and low frame rate.* Each step in the CoELA, COMBO, and MindAgent workloads takes an average of 19.9, 20.8, and 21.4 seconds, respectively. This results in a slow frame and action rate, making real-time execution infeasible for human-agent applications. (2) *LLM-based communication and planning modules dominate latency.* The LLM-based modules contribute significantly to overall mission time, accounting for 90.0% in CoELA (GPT-4-based), 62.5% in COMBO (LLaVA-1.5-based), and 93.7% in MindAgent (GPT-4-based). (3) *Network latency is a significant factor.* For example, network overhead between the agent and the LLM cloud server contributes 27.0% of total latency, significantly impacts total task runtime.

System and collaboration efficiency. We also identify several system-level inefficiencies in cooperative embodied agent systems. (1) *Memory inconsistencies in LLMs during task progression.* Embodied tasks require executing sequential instructions, where later actions depend on earlier ones. As the in-context prompt grows, LLMs struggle to retain critical details, such as previous actions and object locations, leading to errors in long-horizon planning. (2) *Multiple LLM inference runs per execution step.* In CoELA, each step requires three separate LLM runs: message generation (16.1%), planning (36.5%), and action selection (10.3%). This redundancy amplifies inefficiencies, particularly in multi-agent setups and long-horizon tasks. (3) *Sequential processing bottlenecks.* The perception-communication-planning pipeline introduces cumulative latency and redundant high-level planning computations at each step, further slowing down agent actions.

3.3 Cooperative Agents Scalability Analysis

Scalability challenges. While most studies on cooperative embodied agent systems focus on two to four agents, scaling to larger groups presents major challenges, especially in long-horizon, multi-objective tasks: (1) *Exponential growth in coordinated actions and dependencies.* As the number of agents increases, the number of possible coordinated actions and interdependencies grows exponentially, making LLM-based reasoning increasingly complex. (2) *Expanding*

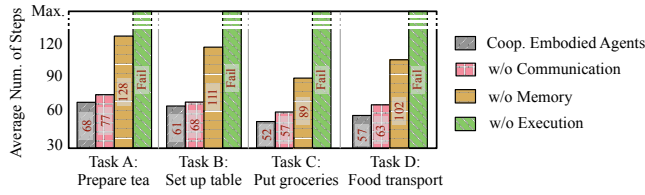


Figure 5. Ablation and sensitivity analysis. The average number of steps required to complete four C-WAH household tasks [63] under different CoELA [86] configurations, highlighting the sensitivity of the memory and execution modules to mission performance.

context length and computational overhead. Each LLM context includes ongoing dialogues, history, actions, and states. More agents result in longer input contexts, approaching LLM token limits, thereby increasing inference runtime and API costs. (3) *Information dilution in long prompts.* Lengthy prompts can dilute critical task-relevant information, reducing the LLM’s ability to generate effective decisions and impacting overall task performance.

Decentralized and centralized comparison. Fig. 4 illustrates the average task success rate and end-to-end latency across varying numbers of agents and tasks. We observe different characteristics of decentralized (CoELA and COMBO) and centralized (MindAgent) architectures in cooperative generalist agent systems. (1) *Decentralized embodied systems suffer from limited scalability and exploded latency.* In decentralized systems, with more agents, the number of communication rounds per planning step grows, often resulting in repetitive and unproductive dialogues, leading to significant latency. Agents frequently reiterate prior suggestions or propose identical actions, which dilutes the context and hampers collaboration. As the number of agents increases, task success initially improves but declines due to reduced collaboration efficiency in larger agent groups. (2) *Centralized systems suffer from sharp task success rate decline under more agents.* In centralized systems, the task success rate sharply decreases as the number of agents grows. This suggests that a single central LLM planner struggles to generate effective plans for complex multi-agent tasks, leading to suboptimal performance in more intricate scenarios.

3.4 Ablation and Sensitivity Analysis

To evaluate the sensitivity of individual modules within the cooperative agent framework (Fig. 1) for long-horizon tasks, we analyze the average number of steps taken to complete various embodied tasks under different configurations.

Communication module sensitivity. The communication module facilitates information exchange and request handling among agents. Surprisingly, we observe from Fig. 5 that disabling communication among agents had no significant impact on performance. We hypothesize two possible reasons: *System-level factors*, pre-generated messages cover agent-relevant interactions, yet fewer than 20% of these are

actually exchanged, suggesting sparse communication usage. *Model-level limitations*, effective communication requires understanding agent intentions and managing natural language ambiguities, which current models like GPT-4 struggle to handle consistently [86].

Memory module sensitivity. Fig. 5 highlights the critical role of memory in cooperative embodied systems. Disabling the memory module results in a 1.84× increase in steps required to complete long-horizon tasks across four benchmarks. This demonstrates that memory is essential for tracking environmental knowledge and agent actions, significantly improving task efficiency.

Execution module sensitivity. The low-level execution module plays an indispensable role in system functionality. As shown in Fig. 5, disabling it led to task failures and reaching the maximum step limit. This likely occurred because, without this module, the LLM-based planning system was forced to handle low-level control decisions, significantly expanding the decision space and slowing inference. This underscores the importance of using LLMs for high-level planning while relying on optimization-based algorithms (e.g., A-star) for low-level control. Developing agents with efficient low-level control is essential for advancing scalable and robust cooperative systems.

4 Integrated Acceleration Overview

While cooperative embodied autonomous agent systems offer significant potential, they face challenges of long task latencies, inefficient collaboration, limited scalability, and sensitivity in low-level planning (Sec. 3). Addressing its inefficiencies is critical for achieving real-time and scalable deployment and ensuring long-term development.

To this end, we introduce ReCA, an integrated acceleration framework designed to enable real-time and efficient cooperative embodied autonomous agent systems (Fig. 6). At the algorithm level, local LLM model processing and planning-guided multi-step reduce the planning and communication latency (Sec. 5.1, Sec. 5.4). At the system level, the hybrid dual-memory structure (Sec. 5.2) and hierarchical centralized/decentralized cooperative planning (Sec. 5.3) enhance cooperative efficiency and system scalability. At the hardware level, the A-star processing unit interacting with the GPU subsystem improves low-level planning and execution efficiency (Sec. 6). ReCA consistently demonstrates improvements in performance, efficiency, and scalability across cooperative embodied systems (Sec. 7).

5 ReCA: System Optimization

This section presents ReCA system optimizations for cooperative embodied agent systems. ReCA features efficient local model processing (Sec. 5.1), dual long/short-term memory structure (Sec. 5.2), hierarchical decentralized-centralized planning (Sec. 5.3), and planning-guided execution (Sec. 5.4).

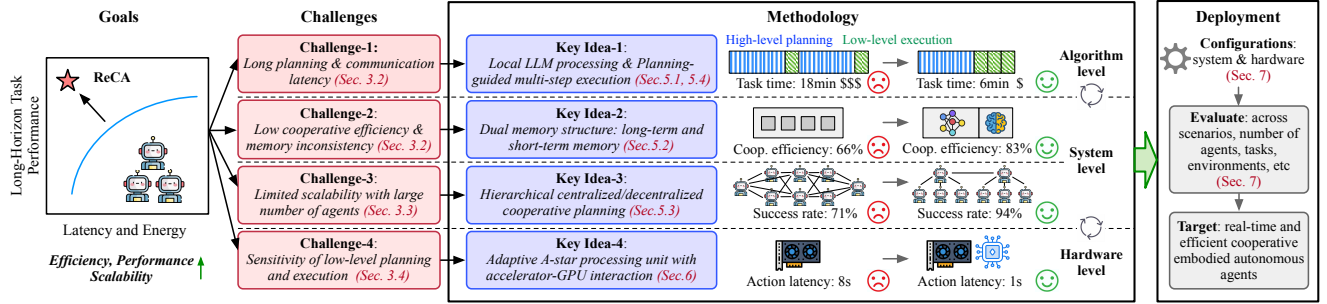


Figure 6. ReCA System Overview. ReCA is an integrated acceleration framework with the goal to enable real-time and efficient cooperative embodied autonomous agent systems. ReCA addresses the challenges of long planning and communication latency, low cooperative efficiency, limited scalability, and low-level planning and execution sensitivity, by proposing methodologies including dual-memory structure, hierarchical cooperative planning, multi-step execution, and hardware subsystems. ReCA is deployed across embodied workloads and scenarios and consistently demonstrates performance-efficiency-scalability improvements for cooperative embodied systems.

5.1 Efficient Local Model Processing

Motivation. ReCA aims to achieve real-time efficiency in generalist multi-agent embodied AI systems. Recent systems [20, 27, 86] have relied heavily on closed-source models such as GPT-4 [57] and Claude-3 [1] for planning and communication due to their superior performance. However, these models pose challenges for optimizing embodied systems. *First, unpredictable API call runtime.* API call performance varies with real-time demand and server usage, making it hard to reduce the latency of LLM requests, which are typically the most time-consuming component in embodied AI systems (Sec. 3.2). *Second, substantial model cost for embodied agents.* The cost of using higher-capability models rises significantly, creating budget constraints that limit the large-scale deployment of embodied AI systems. *Third, privacy concerns are critical* in scenarios involving ubiquitous human-agent interaction, as the use of external API calls introduces risks to data privacy. To address these issues, ReCA explores the feasibility of using smaller, open-source LLMs deployed locally on agents while maintaining comparable performance in long-horizon tasks without API dependencies. Local deployment eliminates the need to transmit sensitive data to external servers, reducing the risk of private data exposure while also minimizing dependency on network latency and API availability.

Model optimization. To optimize the deployment of smaller LLMs in cooperative embodied systems, ReCA employs several techniques to enhance efficiency, scalability, and adaptability: *First, parameter-efficient fine-tuning.* We apply LoRA [25] to fine-tune smaller models using agent-collected datasets, achieving promising performance while minimizing computational overhead. *Second, chain-of-thought (CoT) prompting.* We adopt CoT prompting [78], which breaks down complex tasks into smaller, sequential subtasks, making them more manageable for lightweight LLMs. *Third, hierarchical prompt structuring.* We format prompts using structured hierarchical representations to improve clarity,

consistency, and interpretability for smaller models. *Fourth, optimized model compilation.* We integrate MLC-LLM [68], which compiles models into hardware-optimized representations, leveraging software stack accelerations to enhance execution efficiency. *Finally, model quantization.* We employ AWQ quantization [41] to further reduce model size, enabling efficient deployment on resource-constrained embodied agents without compromising functionality.

Model experimentation. The trade-off between model inference time and task step count directly affects overall mission latency. Simply reducing model size can degrade planning quality, leading to longer task completion times. To strike the optimal balance between model size and task performance, we conduct comprehensive experiments with models of varying sizes (e.g., Llama2-7B, Llama3-8B-Instruct, Llama3.2-1B, Llama3.2-3B, Llama3.1-8B, etc.) to minimize end-to-end task latency by optimizing runtime efficiency and the number of task steps.

5.2 Dual Memory Structure

Motivation. The memory module is critical in ensuring the efficiency and functionality of cooperative embodied agent systems. However, as shown in Sec. 3.2, existing systems struggle with hallucinations and inconsistencies, particularly in long-horizon, multi-objective tasks. These challenges stem from poor long-term retention and difficulty retrieving relevant information when needed. Additionally, permanent memory storage increases storage demands, while resetting memory on restart erases long-term knowledge, leading to performance degradation. Therefore, optimizing the memory module is essential for enhancing agent performance.

Dual memory structure overview. To address memory inconsistencies and enhance long-horizon planning capabilities, ReCA introduces a dual-memory structure, which divides the memory module into two complementary components: long-term memory and short-term memory (Fig. 7). Long-term memory stores static environmental information such as objects, layouts, and structural features, while

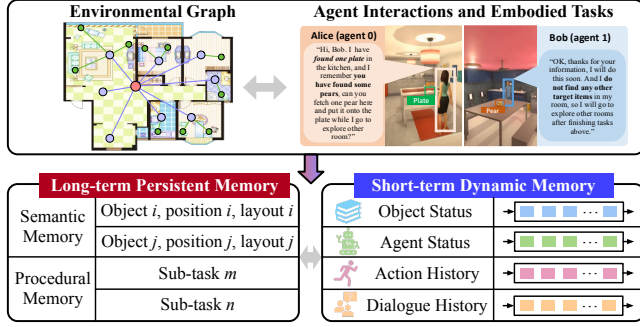


Figure 7. Dual memory structure. ReCA presents a dual memory structure consisting of long-term memory and short-term memory to improve high-level planning and cooperative efficiency.

short-term memory tracks real-time updates on agent status, task progress, and recent interactions. This structure enables agents to combine stable environmental knowledge with immediate task-specific details, improving their performance in complex multi-step tasks.

Long-term memory. Long-term memory is designed to store static environmental information, including layout structures and static object positions, which remain unchanged over extended periods. To enable efficient management and retrieval, perceptual data is structured as a hierarchical graph, encoding spatial relationships and object attributes. As illustrated in Fig. 7, this graph represents objects as vertices and their relationships as edges, providing a structured and scalable representation of the environment.

As agents cooperatively navigate the environment, they dynamically construct and update this graph, expanding it as new areas are explored. This enables agents to efficiently integrate spatial knowledge while continuously refining their understanding. When the LLM planner requires environmental data, the graph is serialized into a structured text format for direct parsing. This approach minimizes redundant exploration and enables fast updates by dynamically adding or removing nodes based on environmental changes. As a result, agents can make well-informed decisions, improving long-horizon planning efficiency and enhancing accuracy.

Short-term memory. Short-term memory is compact and dynamically refreshed, storing recent task-related information such as object states, agent status, and interaction history. This ensures quick access to recently used objects and their attributes, streamlining task execution.

To optimize LLM planning efficiency, short-term memory is structured for serialization into text format. This allows the planner to seamlessly incorporate recent observations and task progress into decision-making. For instance, in the Fig. 2 scenario, if an agent is instructed to “fetch a pear and place it on a plate,” its location and status are logged in short-term memory. When a subsequent instruction requires finding the same object, the agent can retrieve this stored data instead of re-exploring the environment, significantly reducing redundant operations and improving efficiency.

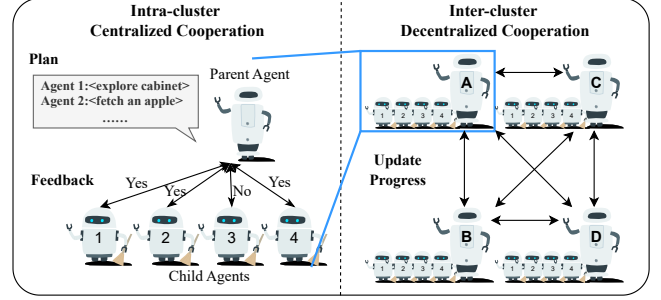


Figure 8. Hierarchical cooperative planning. ReCA presents a hierarchical paradigm featuring parent-child centralized cooperation within clusters and decentralized cooperation across clusters.

Throughout task execution, short-term memory is dynamically updated by replacing the least recently used data with the most recent information. This adaptive retention strategy ensures that critical, high-priority data remains readily accessible, while outdated or less relevant information is systematically discarded. By prioritizing task-relevant details over retrieving entire dialogue histories, this approach prevents context dilution, optimizes memory efficiency, and enhances overall task performance.

5.3 Hierarchical Cooperative Task Planning

Motivation. As the number of embodied agents increases, both fully centralized and fully decentralized cooperative embodied systems encounter scalability challenges (Sec. 3.3). Decentralized systems suffer from increased latency and reduced planning efficiency, while centralized systems experience a sharp decline in task success rates. These limitations necessitate a more scalable approach to ensure efficient task execution in large-scale cooperative systems.

Hierarchical cooperative planning. To overcome these challenges, ReCA introduces a hierarchical cooperative planning paradigm, as illustrated in Fig. 8. In this paradigm, agents are grouped into clusters when they are close enough to interact. Within each cluster, ReCA utilizes a parent-child *centralized* cooperation model, while coordination between clusters follows a *decentralized* approach.

Intra-cluster centralized cooperation. Within each cluster, the parent agent acts as the central planner, leveraging its LLM to generate task plans for both itself and its child agents. The parent communicates these plans to child agents and other parent agents across clusters. Upon receiving their assigned actions, child agents review them and provide feedback. If any action is deemed infeasible, the parent agent re-plans accordingly. This iterative feedback loop continues until all agents in the cluster agree on their respective actions, which can enhance task success rates and eliminate invalid actions in large-scale cooperative systems. Agents execute their tasks asynchronously, dynamically resolving dependencies during execution. Cluster-wide synchronization occurs after each round of actions, minimizing workload

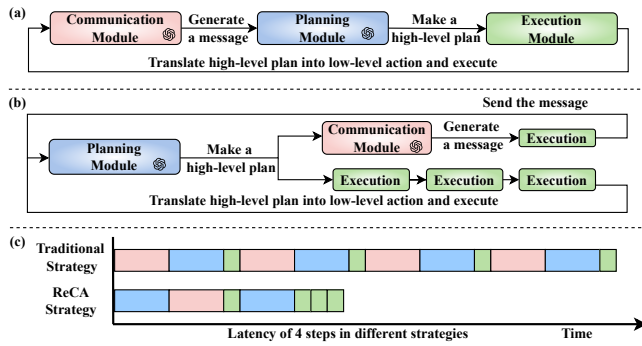


Figure 9. Planning-guided efficient execution. ReCA presents a planning-then-communication strategy and planning-guided multi-step execution, effectively decoupling high-level and low-level planning to significantly reduce end-to-end system latency.

imbalances and enhancing overall system efficiency by avoiding the overhead of global synchronization across all agents.

Inter-cluster decentralized cooperation. For interactions between clusters, parent agents from each cluster communicate to update task progress while minimizing communication overhead and context dilution among agents. Cluster composition is dynamically adjusted based on the evolving distances between agents. To mitigate computational overhead in systems with a large number of agents (e.g., N agents), ReCA sweeps the number of agents per cluster (N_C) based on the deployed workload to identify the optimal cluster size (N'_C). It then designates $\lceil N/N'_C \rceil$ agents as parent agents, each leading a separate cluster, thereby distributing the planning workload more efficiently.

Dynamic task decomposition and allocation. The LLM-based planner within each parent agent handles task decomposition and sub-task allocation. By leveraging the overall mission goal, semantic map from long-term memory, and agent/object status from short-term memory, the planner generates optimized sub-task sequences. It considers the spatial distribution of agents and objects to allocate tasks in a manner that minimizes overall execution time. Throughout the mission, task decomposition and allocation remain dynamic, with parent agents adjusting plans in response to new observations and updated agent statuses. This adaptability enables the system to respond effectively to environmental changes and optimize task performance in real time.

5.4 Planning-Guided Efficient Execution

Motivation. Inefficiencies in execution and collaboration significantly hinder the real-time performance of cooperative embodied agents (Sec. 3.2). From a communication perspective, the message pre-generation process introduces unnecessary redundancy as depicted in Fig. 9a. Messages are generated at every step for each agent, yet only around 20% of these steps result in actual communication once plans are finalized. From a planning perspective, the sequential execution model, which requires high-level planning decisions

from the LLM before each low-level action, leads to substantial runtime latency, adversely affecting system efficiency.

Planning-then-communication strategy. To mitigate these issues, ReCA adopts a planning-then-communication strategy that optimizes communication by tightly integrating it with the planning process. Instead of pre-generating messages at every step, the planning module assesses the need for communication based on data from the dual memory architecture, which stores environmental information, agent states, actions, and dialogue history. Messages are generated only when communication is essential, thereby eliminating redundancy and reducing overhead.

Planning-guided multi-step execution. To further reduce end-to-end latency, ReCA redefines the execution pipeline for cooperative embodied AI systems, shifting away from the conventional step-by-step high-level planning approach. Instead of generating a high-level plan for each low-level action, ReCA allows the planning module to produce a high-level plan that guides multiple consecutive low-level actions within a specified time frame.

Multi-step execution may introduce cumulative errors. To mitigate their impact, ReCA employs two key strategies: (1) Pre-deployment optimization, sweeping the number of multi-step executions to identify the system's optimal balance between success rate and task latency. (2) Runtime adaptation, utilizing a self-reflection module (rule-based or LLM-based) to continuously monitor agent statuses, trigger re-planning when necessary, and update accurate operational information in the memory module.

By implementing these strategies, along with the newly designed execution pipeline shown in Fig. 9b, ReCA reduces the frequency of LLM inferences, leading to substantial savings in both latency and energy, as shown in Fig. 9c. This approach minimizes the need for frequent high-level planning, allowing agents to execute multiple actions continuously without additional LLM input. As a result, the latency bottleneck shifts from LLM-based planning to the efficient execution of low-level actions, making the optimization of the low-level action module essential for achieving real-time performance in embodied AI systems.

6 ReCA: Hardware Architecture

This section presents ReCA system architecture designed to support cooperative embodied agents. ReCA integrates a high-level LLM subsystem and low-level A-star accelerator subsystem (Sec. 6.1), with efficient support across computation blocks (Sec. 6.2), processing units (Sec. 6.3), and heterogeneous subsystem interface (Sec. 6.4).

6.1 Hardware Subsystems Overview

Cooperative embodied autonomous agents employ hierarchical high-level planning and low-level execution to accomplish long-horizon, multi-objective tasks. With optimizations

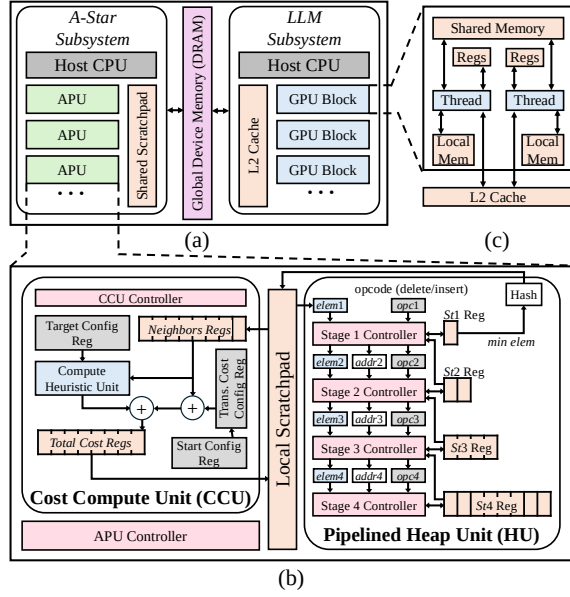


Figure 10. Embodied system hardware acceleration. (a) Integrated system architecture, including both the LLM subsystem and the A-star subsystem. (b) APU (A-star Processing Unit) architecture. (c) GPU block architecture.

on high-level functions and multi-step execution (Sec. 5), the efficiency and scalability of low-level planning become even more critical to system performance (Sec. 3.4). To further accelerate cooperative embodied AI workloads, ReCA is empowered by a system architecture that is built around two primary subsystems: the A-star Subsystem and the LLM Subsystem (Fig. 10a). The A-star Subsystem is designed to execute low-level planning tasks through A-star search algorithms. The core component of this subsystem is the *A-star Processing Unit* (APU), a specialized hardware module optimized for state-space search computations. The LLM Subsystem, on the other hand, handles the LLM inferences. It is composed of GPU optimized for matrix multiplications and Transformer-based operations. Each subsystem operates under the control of a host CPU.

Agent-Specific Architecture. The proposed architecture is adaptable to various agent types, particularly the hierarchical parent and child embodied agents paradigm (Sec. 5.3). Parent agents, responsible for high-level planning and decision-making, integrate both the A-star subsystem and LLM subsystem. Child agents utilize a streamlined architecture with the A-star subsystem, focusing on efficiently and robustly executing low-level plans assigned by the parent agents.

6.2 A-Star Path Planning Computations

A-Star Acceleration Motivation. A-star is a widely used algorithm for path planning in robotics and embodied systems due to its optimality and completeness under certain conditions [4, 5, 31, 43, 62], and its performance is critical to the success rate of embodied systems (Fig. 5). While A-star

accounts for 5-15% of the latency in the original system, this attribute can increase to around 20–40% after optimizing LLM-based modules, making it a system bottleneck. Moreover, A-star is highly inefficient on off-the-shelf CPUs and GPUs due to its interactive and sequential nature, limited parallelism, high branching, and frequent memory access. These challenges highlight the need for system- and architecture-level optimizations to reduce A-star latency, making a custom A-star architecture essential for efficient low-level planning and execution in embodied systems.

A-star workflow. For clarity, the A-star algorithm is illustrated using a 2D grid search space, where each cell represents a possible state the robotic agent can occupy (Fig. 11a). The objective is to move from a start cell (S) to a goal cell (F) while minimizing the total path cost. Each cell is assigned two cost metrics: the (g) cost, which represents the exact cost of reaching that cell from the start, and the (h) cost, a heuristic estimate of the remaining distance from the current cell to the goal. The algorithm computes the total cost for each cell as $f = g + h$, and this value is used to prioritize the cells that appear most promising for achieving the goal.

The path planning algorithm maintains a frontier of candidate cells in a priority queue (min heap) (Fig. 11b). In the initial step, the start cell (S) is inserted into the heap, and its f value is calculated using $g = 0$ and the estimated h value based on the distance to the goal. The heap dynamically orders the cells by their f values, ensuring that the cell with the lowest cost is processed first. During each iteration, the cell with the smallest f value is removed from the heap, marking it as the current cell for expansion. For each neighboring cell, the path planning calculates the g and h costs and inserts them into the heap if they have not yet been explored or updates their costs if a more efficient path is found. The algorithm repeats these steps until it reaches the goal cell or exhausts all possible paths.

Key computation blocks. The computational flow of A-star consists of several critical operations performed at each step (Fig. 11c). These operations include deleting the lowest-cost cell from the min heap, expanding the selected cell to retrieve neighboring cells, computing their costs, and reinserting or updating neighbors in the heap based on their new f values. Efficient execution of these operations is crucial, especially for large grids or complex search spaces. This motivates the need for hardware acceleration to optimize two key areas: min heap management and cost computation. Specialized hardware for heap operations, such as insertion, deletion, and reordering, can significantly reduce processing time, while parallelized units for calculating g , h , and f values enable faster cost evaluation. Additionally, an efficient memory mechanism is essential to quickly access and update neighboring cells, minimizing latency during node expansion. Together, these hardware enhancements provide the necessary speedup for real-time, low-level path-planning tasks in cooperative embodied systems.

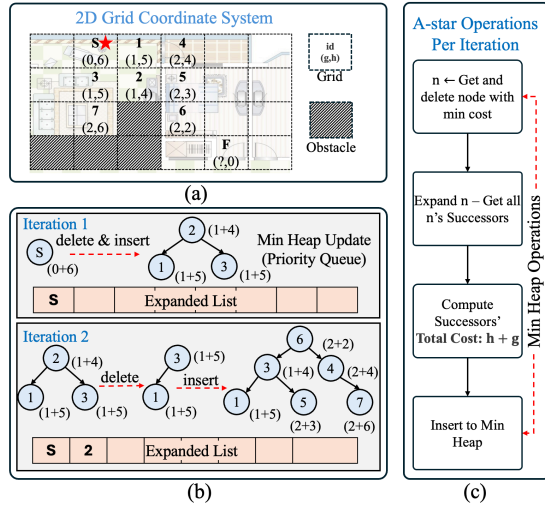


Figure 11. A-star planning computations. (a) Example planning problem mapped onto a 2D grid coordinate system. (b) Algorithmic representation and operations of min heap. (c) A flowchart summarizing A-star computations in each iteration.

6.3 A-star Processing Unit (APU)

The APU subsystem optimizes key computational tasks in the A-star planning algorithm through specialized hardware for cost computation, heap management, and memory access. As illustrated in Fig. 10b, its architecture comprises three main components: the *Cost Compute Unit* for cell cost evaluation, the *Heap Unit* for priority queue management, and a *scratchpad memory* for storing neighboring cell data during node expansion. These components collaboratively accelerate A-star operations, reducing delays from repetitive arithmetic and memory access. Iteratively, the heap extracts the lowest-cost cell, the cost compute unit computes neighbor costs, and updates are made to both the heap and scratchpad for the next iteration.

Cost compute unit. The cost compute unit is responsible for calculating the g , h , and f values for the neighboring cells of the current node. It includes dedicated arithmetic units that can handle multiple operations in parallel, significantly speeding up the computation process. The cost compute unit takes the current cell's coordinates, cost information, and heuristic input, computes the new values for each neighbor, and outputs the results for insertion into the scratchpad memory.

Heap unit. The heap unit manages the priority queue (min heap) of open cells based on their f values. Optimized for efficient operations such as insertion, deletion, and re-ordering, the heap unit dynamically updates the heap as new cells are added or when existing cells have their costs recalculated. A specialized four-level control circuit ensures that the heap remains balanced, enabling the extraction of the lowest-cost cell (highest priority) with minimal overhead for real-time path exploration. The heap unit is designed with a

single interface point at the first-level controller, eliminating the need for manual control of the deeper hardware circuits.

Scratchpad memory. The scratchpad memory within the APU serves as fast short-term storage, supporting the iterative nature of the A-star path planning algorithm. Acting as a temporary buffer, it holds critical data during node expansions, including metadata related to neighboring cells (or search subspace) and the computed cost values for expanded nodes. A dual-port memory design allows simultaneous read and write operations, enabling the APU to quickly retrieve and update cost values during each iteration.

At the subsystem level, the global scratchpad memory provides a larger, shared space for managing data that persists across multiple iterations (or processes) of the A-star planning and facilitates communication between the APUs and the host CPU. Unlike APU's local scratchpad, the global scratchpad serves as a long-term memory that stores more comprehensive information about the entire environmental space, including metadata on planning and search progress and the history of visited nodes.

APU data and control flows. The APU accelerates path planning by efficiently orchestrating data and control flows through its core components: cost compute unit, heap unit, and scratchpad memory. The process begins with loading search parameters, such as start and target coordinates, into the APU's local scratchpad and configuration registers via the global scratchpad. These parameters serve as fixed references guiding the iterative execution of the A-star search path planning procedure.

The control flow initiates with the heap unit constructing the multi-level priority queue by inserting the starting node (S in Fig. 11) along with its initial f (total cost) value. The heap unit autonomously extracts the lowest-cost node from the heap to be expanded, which is S in the first iteration. Once a node is selected, its neighboring cells are identified, and their corresponding data (coordinates and current costs) are fetched from the scratchpad memory. The cost compute unit then performs parallel calculations of g , h , and f values for each neighbor using dedicated arithmetic units. This parallel processing of multiple neighbors significantly accelerates the expansion step.

An efficient dataflow design is critical for rapid neighbor extraction. The identification of neighboring cells relies on how the scratchpad stores data, ensuring that the spatial locality of cells in the search space is reflected in their storage layout. Specifically, when the current cell's index is provided, the scratchpad uses hardware-implemented hash function as an indexing mechanism to generate the addresses of neighboring cells. The hash function computes the scratchpad memory addresses corresponding to the neighboring cell based on the spatial coordinates of the current cell, allowing the APU to quickly fetch the relevant data for each neighbor without needing to search through the entire scratchpad. This locality-based addressing mechanism accelerates the

node expansion process by ensuring that neighboring cells can be accessed with minimal delay.

After the cost compute unit computes the costs, the control flow directs the insertion of the new f values into the heap unit and updates the local scratchpad with the neighbor information. The heap unit then reorders the priority queue based on these new costs, ensuring that the lowest-cost node is always selected for the next iteration. The dual-port design of the scratchpad memory allows simultaneous read and write operations, thus reducing memory access latency. This iterative cycle of node extraction, neighbor expansion, and heap updating continues until the target node F is reached.

The APU’s architecture is scalable and generalizable, capable of supporting any n -dimensional path planning tasks. The modular design of the cost compute unit and heap unit allows the system to handle more complex environments, from 2D grids to n -dimensional spaces. The parallelism within the cost compute unit and efficient heap management in the heap unit ensure that the APU can adapt to varied complexity levels in path planning tasks. The local and global scratchpad memories further enhance its scalability, enabling efficient data management across large search spaces.

6.4 Heterogeneous Subsystem Interface

The proposed design features a heterogeneous architecture where the A-star subsystem and LLM subsystem interface through shared global DRAM, as illustrated in Fig. 10a. Data transitions between the two subsystems are structured based on their respective roles: the LLM subsystem writes high-level plans to memory, which the A-star subsystem retrieves and executes for low-level planning. The A-star subsystem reads this data, configures its search parameters through Start and Target Configuration Registers, and upon execution, returns feedback (such as search outcomes or error conditions) to DRAM for further processing by the LLM subsystem. This bidirectional data flow enables adaptive planning, allowing the LLM subsystem to refine its planned actions and strategies based on the low-level execution results from the A-star subsystem.

Synchronization between two subsystems is managed by the host, which coordinates data transfers and ensures consistency through memory flushes, barriers, and interrupts. The programming model for the APU subsystem involves the GPU command buffer [19] configuring the APU and scheduling the low-level path planning and execution module on the APU subsystem. This architectural design facilitates efficient communication between the subsystems while supporting the varied requirements of parent and child agents. In child agents, the interface is streamlined: the global DRAM primarily serves to fetch task-specific configurations (start/target) for the APU from external sources or parent agents. Since child agents rely on parent agents’ high-level LLM planning results within the cluster, the DRAM facilitates direct low-level planning execution without high-level LLM inference.

7 Evaluation Results

This section first introduces the detailed settings for evaluating our proposed ReCA framework (Sec. 7.1), and then benchmarks the performance and efficiency of our proposed ReCA optimization techniques (Sec. 7.2) with sensitivity analysis (Sec. 7.3), demonstrating the practical of efficient and scalable cooperative embodied AI systems.

7.1 Experimental Setup

Workloads. We evaluate ReCA on three state-of-the-art cooperative embodied systems, i.e., CoELA [86] COMBO [87], and MindAgent [20]. We follow the configurations and parameters reported in their studies to set up baseline systems.

Benchmarks. We evaluate ReCA on six commonly-used cooperative embodied AI benchmarks, i.e., TDW-MAT [15], TDW-Cook [14], TDW-Game [14], CuisineWorld [20], C-WAH [63], and Minecraft [22]. These benchmarks include collaborative objective transporting, housework, gaming, etc long-horizon multi-objective planning tasks.

Metrics. To evaluate the performance and efficiency of the ReCA system, we report the average task success rate, number of steps, and end-to-end runtime. The success rate is the percentage of successful trials where a successful mission is achieved when all agents finish their subtasks within upper step limits. The average number of task steps and end-to-end runtime are reported for successful missions.

Network. We assume all embodied agents run on local machines and are interconnected via WAN. Based on real-world infrastructure, we assume a WAN latency of 75 ms and a throughput limit of 100 Mbps.

Hardware. We evaluate the LLM inference runs latency and energy consumption on Nvidia A6000 GPU, measuring power with NVML [69]. Control latency and power are measured using an Intel i7 CPU. We implement ReCA A-star subsystem on the Xilinx Zynq-7000 SoC ZC706 FPGA [80] to assess real hardware performance.

7.2 ReCA Performance and Efficiency

Success rate improvement. Fig. 12a shows the average task success rate of cooperative embodied AI agent systems across six benchmarks. We use the original workload implementations with four embodied agents as the baseline. ReCA uses the Llama-3.1-8b model along with other system and hardware optimizations. We observe that ReCA consistently improves the success rate by an average of 4.3% across long-horizon multi-objective tasks. This improvement is mainly attributed to enhanced memory consistency via dual-memory structure and improved cooperation capability via the hierarchical decentralized-centralized mechanism.

Efficiency improvement. Fig. 12b shows the average end-to-end task runtime and the number of steps required to complete the task of cooperative embodied AI agent systems. We observe that ReCA requires only 3.2% more steps, but

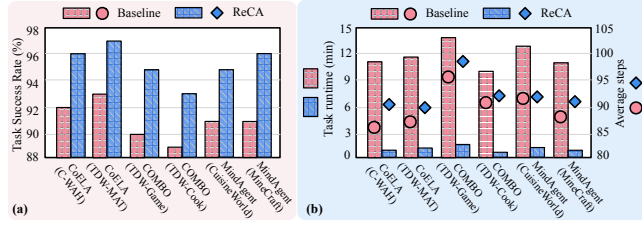


Figure 12. Success rate and efficiency improvement. ReCA (a) improves the task success rate and (b) reduces end-to-end task latency across six complex long-horizon multi-objective tasks.

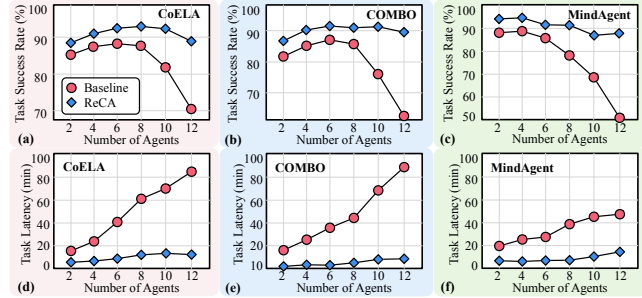


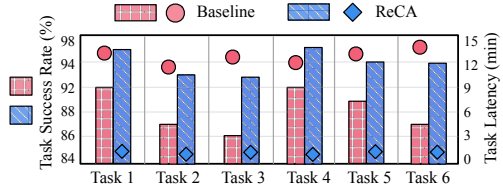
Figure 13. Scalability improvement. ReCA improves the system scalability with improved task success rates and reduced end-to-end latency under large-scale cooperative embodied agent systems.

delivers a 10.2 $\times$ average speedup over the original CoELA, COMBO, and MindAgent systems. This is primarily due to reduced latency per step. We also conduct ten user studies for human-agent interaction and observe a 3.3 min average task completion time. ReCA, on the contrary, achieves a 1.2 min task runtime, enhancing system efficiency and real-time performance.

Scalability improvement. Fig. 13 illustrates the average task success rate and end-to-end latency under different numbers of embodied agents. We observe that ReCA consistently improves cooperative long-horizon task performance and efficiency, compared to both fully decentralized systems (CoELA and COMBO) and fully centralized systems (MindAgent). For example, under 12 embodied agents, the baseline systems typically take 1 hour to finish complex cooperative tasks with only <70% success rate, whereas ReCA can finish them in within 20 minutes and maintain a 90% success rate.

Effectiveness across tasks. Fig. 14 evaluates ReCA on six different C-WAH tasks with varying difficulty levels. We observe that ReCA is adaptive across tasks, with an average of 4.0% success rate improvement and 9.8 $\times$ speedup across six long-horizon object transporting tasks.

Hardware resource consumption. The ReCA A-star architecture is compact and resource-efficient, making it well-suited for deployment on real-world resource-constrained physical agents. When implemented on Xilinx ZC706 FPGA, the design utilizes 25.3% of digital signal processors (DSPs), 15.6% of flip-flops (FFs), and 21.1% of look-up tables (LUTs). Additionally, the optimized custom scratchpad enables the



Task	Descriptions
1	Find and place 3 forks and 1 plate into the dishwasher
2	Find and place 1 bottle of wine, 1 pancake, 1 pound cake, 1 juice, and 1 apple on the kitchen table
3	Find and place 3 forks into the dishwasher
4	Find and place 1 pudding, 1 juice, 1 apple, and 2 cupcakes on the coffee table
5	Find and place 1 bottle of wine, 2 cupcakes, and 1 pudding on the coffee table
6	Find and place 1 bottle of wine, 1 juice, 1 apple, 1 cupcake, and 1 pound cake on the kitchen table

Figure 14. Effectiveness across tasks. ReCA consistently improves the task success rate and efficiency across six C-WAH tasks with varying difficulty levels.



Figure 15. Sensitivity across multi-step execution. Task success rate and latency across scenarios of different numbers of low-level action steps agents take after high-level planned instructions.

ReCA A-star accelerator to consume only 10.7% of the total block random access memory (BRAM).

Energy consumption. The ReCA heterogeneous hardware system efficiently processes high-level LLM planning (via GPU subsystem) and low-level planning and control (via APU subsystem). For the low-level A-star planning module, we observe that ReCA APU subsystem achieves 4.6 $\times$ speedup compared to the GPU-based A-star implementation while delivering a 281 $\times$ improvement in energy efficiency.

7.3 Sensitivity Analysis

Sensitivity across multi-step execution. Fig. 15 evaluates the performance and efficiency of ReCA across various multi-step execution scenarios (Sec. 5.4), specifically examining the number of low-level planning and action steps agents take after receiving each high-level LLM-based instruction. We observe that increasing the number of consecutive low-level action steps initially reduces end-to-end task latency by minimizing the number of LLM runs. Beyond a certain point, the total number of steps required to complete the task rises sharply due to accumulated trajectory errors, leading to increased overall latency and decrease in task success rate.

Sensitivity across hierarchical cooperative planning. Fig. 16 evaluates the performance and efficiency of ReCA under different hierarchical cooperative planning configurations (Sec. 5.3), focusing on the average number of agents per cluster. We observe that both too few or too many agents per cluster negatively impact task success rates and increase end-to-end latency, particularly in large-scale systems. Having too few agents per cluster causes the system to behave more

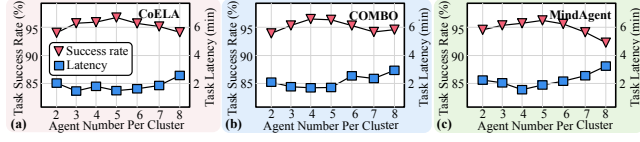


Figure 16. Sensitivity across hierarchical cooperative planning. Task success rate and latency across scenarios of different number of agents in each cluster during centralized intra-cluster and decentralized inter-cluster cooperation.

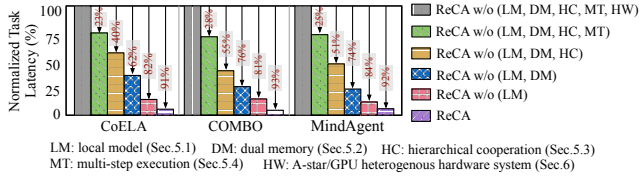


Figure 17. Ablation study on ReCA optimization techniques. The task latency achieved by ReCA w/o the local model, dual memory, hierarchical cooperation, multi-step execution, and heterogeneous hardware systems across embodied AI tasks.

like a decentralized architecture, while having too many agents leads to centralized behavior. These observations further validate the advantages of our proposed hierarchical approach, which combines decentralized inter-cluster cooperation with centralized intra-cluster planning.

Ablation study on the proposed optimization techniques. As illustrated in Sec. 5 and Sec. 6, ReCA features a software-system-hardware co-design technique aimed at enhancing the efficiency (latency), performance (success rate), and scalability (agent number capacity) of cooperative embodied systems. The framework consists of efficient local model processing, dual-memory structure, hierarchical communication, planning-guided multi-step execution, and GPU/A-star heterogeneous hardware system to improve cooperative embodied task efficiency and performance. To verify the effectiveness of our proposed methods, we summarize the end-to-end task latency and success rate of ReCA w/o each technique in Fig. 17. In particular, the proposed GPU/APU hardware system can trim down the runtime by 25.3% on average. Additionally, with the proposed multi-step execution, hierarchical planning, and dual-memory structure, the runtime reduction ratio can be further enlarged to 48.7%, 70.7%, and 82.3%, indicating that both proposed techniques are necessary for ReCA to achieve the desired efficient and scalable cooperative embodied intelligence.

8 Related Work

Embodied AI system. Embodied AI systems integrate perception, cognition, and action, enabling agents to interact with the physical world and perform complex tasks [13, 28, 81]. These systems process sensory inputs, reason about their surroundings, and execute actions by leveraging cognitive models for planning and control [48, 50, 66, 76, 77, 84, 85]. Typically, they consist of a server equipped with GPUs for

LLM execution and a physical agent for interaction [28], where real-time responsiveness is critical. Integrated task and motion planning enhances decision-making by combining task reasoning with motion control [2, 74], while model-based reasoning and reinforcement learning help agents manage uncertainty and adapt to long-horizon tasks [70]. ReCA is designed for embodied physical agent systems that integrate hierarchical high-level and low-level planning, providing a structured framework that includes perception, memory, communication, and execution modules.

Multi-agent collaboration. Multi-agent cooperation involves agents working together to achieve shared goals via coordination, communication, and collaboration. Research has explored various mechanisms to synchronize actions, mitigate conflicts, and enhance overall efficiency [11, 12, 21, 26, 29, 30, 43, 61, 75, 83, 88, 90]. Collaboration techniques enable agents to leverage each other’s strengths, drawing on algorithms from distributed systems, game theory, and cooperative control to optimize performance [49, 53, 60]. ReCA is designed to enhance real-time efficiency in cooperative long-horizon multi-objective tasks.

Efficient autonomous systems. Recent advancements in autonomous system design span multiple domains, including domain-specific languages [61, 64], simulation tools [8, 35, 56], benchmark suites [37, 50], runtime systems [9, 65], design frameworks [7, 23, 36, 38, 54, 55], and safety considerations [24, 72]. Additionally, optimized compute units and accelerators have been developed for both FPGA [3, 16, 17, 44–46, 51, 52, 52, 73, 82] and ASIC implementations [6, 10, 32, 39, 40, 58, 67]. ReCA complements these techniques and can be integrated with them to enable real-time, efficient cooperative embodied systems. While demonstrated in generalist agents, ReCA’s acceleration techniques are broadly applicable to a wide range of single- and multi-agent systems.

9 Conclusion

To enable efficient and scalable cooperative agent systems for real-time embodied intelligence, we present ReCA, the integrated software-hardware co-design framework dedicated to accelerating cooperative embodied systems. ReCA identifies opportunities for accelerating cooperative embodied AI, including dual memory structure, hierarchical cooperative planning, planning-guided multi-step execution, and scalable A-star accelerator. We believe ReCA paves the way for a new perspective on scalable and efficient cooperative embodied agent systems.

Acknowledgments

We thank the anonymous ASPLOS reviewers for their thorough comments and feedback. This work was supported in part by CoCoSys, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA, and NSF GRFP DGE-214074.

References

- [1] Anthropic. Claude 3. <https://www.anthropic.com/index/claude-3>, 2024. Accessed: 2024-10-06.
- [2] Luke Antonyshyn, Jefferson Silveira, Sidney Givigi, and Joshua Marshall. Multiple mobile robot task and motion planning: A survey. *ACM Computing Surveys*, 55(10):1–35, 2023.
- [3] Bahar Asgari, Ramyad Hadidi, Nima Shoghi Ghaleshahi, and Hyesoon Kim. Pisces: power-aware implementation of slam by customizing efficient sparse algebra. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.
- [4] Mohammad Bakhshalipour, Seyed Borna Ehsani, Mohamad Qadri, Dominic Guri, Maxim Likhachev, and Phillip B Gibbons. Racod: algorithm/hardware co-design for mobile robot path planning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pages 597–609, 2022.
- [5] Mohammad Bakhshalipour, Maxim Likhachev, and Phillip B Gibbons. Rtrbench: A benchmark suite for real-time robotics. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 175–186. IEEE, 2022.
- [6] Pete Bannon, Ganesh Venkataramanan, Debjit Das Sarma, and Emil Talpes. Computer and redundancy solution for the full self-driving computer. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, pages 1–22. IEEE Computer Society, 2019.
- [7] Kshitij Bhardwaj, Zishen Wan, Arijit Raychowdhury, and Ryan Goldhahn. Real-time fully unsupervised domain adaptation for lane detection in autonomous driving. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–2. IEEE, 2023.
- [8] Behzad Boroujerdian, Hasan Genc, Srivatsan Krishnan, Wenzhi Cui, Aleksandra Faust, and Vijay Reddi. Mavbench: Micro aerial vehicle benchmarking. In *2018 51st annual IEEE/ACM international symposium on microarchitecture (MICRO)*, pages 894–907. IEEE, 2018.
- [9] Behzad Boroujerdian, Radhika Ghosal, Jonathan Cruz, Brian Plancher, and Vijay Janapa Reddi. Roborun: A robot runtime to exploit spatial heterogeneity. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 829–834. IEEE, 2021.
- [10] Muya Chang, Ashwin Sanjay Lele, Samuel D Spetalnick, Brian Crafton, Shota Konno, Zishen Wan, Ashwin Bhat, Win-San Khwa, Yu-Der Chih, Meng-Fan Chang, et al. A 73.53 tops/w 14.74 tops heterogeneous rram in-memory and sram near-memory soc for hybrid frame and event-based target tracking. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 426–428. IEEE, 2023.
- [11] Yongchao Chen, Jacob Arkin, Yang Zhang, Nicholas Roy, and Chuchu Fan. Scalable multi-robot collaboration with large language models: Centralized or decentralized systems? In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4311–4317. IEEE, 2024.
- [12] Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Mike Rabbat, and Joelle Pineau. Tarmac: Targeted multi-agent communication. In *International Conference on machine learning*, pages 1538–1546. PMLR, 2019.
- [13] Jiafei Duan, Samson Yu, Hui Li Tan, Hongyuan Zhu, and Cheston Tan. A survey of embodied ai: From simulators to research tasks. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 6(2):230–244, 2022.
- [14] Chuang Gan, Jeremy Schwartz, Seth Alter, Damian Mrowca, Martin Schrimpf, James Traer, Julian De Freitas, Jonas Kubilius, Abhishek Bhandwalder, Nick Haber, et al. Threedworld: A platform for interactive multi-modal physical simulation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [15] Chuang Gan, Siyuan Zhou, Jeremy Schwartz, Seth Alter, Abhishek Bhandwalder, Dan Gutfreund, Daniel LK Yamins, James J DiCarlo, Josh McDermott, Antonio Torralba, et al. The threedworld transport challenge: A visually guided task-and-motion planning benchmark towards physically realistic embodied ai. In *2022 International conference on robotics and automation (ICRA)*, pages 8847–8854. IEEE, 2022.
- [16] Yiming Gan, Yu Bo, Boyuan Tian, Leimeng Xu, Wei Hu, Shaoshan Liu, Qiang Liu, Yanjun Zhang, Jie Tang, and Yuhao Zhu. Eudoxus: Characterizing and accelerating localization in autonomous machines industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 827–840. IEEE, 2021.
- [17] Tian Gao, Zishen Wan, Yuyang Zhang, Bo Yu, Yanjun Zhang, Shaoshan Liu, and Arijit Raychowdhury. Ielas: An elas-based energy-efficient accelerator for real-time stereo matching on fpga platform. In *2021 IEEE 3rd International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, pages 1–4. IEEE, 2021.
- [18] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293, 2021.
- [19] Fabian Giesen. A trip through the graphics pipeline 2011: Index. <https://fgiesen.wordpress.com/2011/07/09/a-trip-through-the-graphics-pipeline-2011-index/>, 2011.
- [20] Ran Gong, Qiuyuan Huang, Xiaojian Ma, Hoi Vo, Zane Durante, Yusuke Noda, Zilong Zheng, Song-Chun Zhu, Demetri Terzopoulos, Li Fei-Fei, et al. Mindagent: Emergent gaming interaction. In *Findings of the Association for Computational Linguistics (NAACL)*, pages 3154–3183, 2024.
- [21] Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L Griffiths, and Mengdi Wang. Embodied llm agents learn to cooperate in organized teams. *arXiv preprint arXiv:2403.12482*, 2024.
- [22] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [23] Yuhui Hao, Yiming Gan, Bo Yu, Qiang Liu, Yinhe Han, Zishen Wan, and Shaoshan Liu. Orianna: An accelerator generation framework for optimization-based robotic applications. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 813–829, 2024.
- [24] Yu-Shun Hsiao, Zishen Wan, Tianyu Jia, Radhika Ghosal, Abdulrahman Mahmoud, Arijit Raychowdhury, David Brooks, Gu-Yeon Wei, and Vijay Janapa Reddi. Mavfi: An end-to-end fault analysis framework with anomaly detection and recovery for micro aerial vehicles. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2023.
- [25] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [26] Senkang Hu, Zhengru Fang, Zihan Fang, Xianhao Chen, and Yuguang Fang. Agentscodriver: Large language model empowered collaborative driving with lifelong learning. *arXiv preprint arXiv:2404.06345*, 2024.
- [27] Wenyue Hua, Lizhou Fan, Lingyao Li, Kai Mei, Jianchao Ji, Yingqiang Ge, Libby Hemphill, and Yongfeng Zhang. War and peace (waragent): Large language model-based multi-agent simulation of world wars, 2024.
- [28] Yiyang Huang, Yuhui Hao, Bo Yu, Feng Yan, Yuxin Yang, Feng Min, Yinhe Han, Lin Ma, Shaoshan Liu, Qiang Liu, et al. Corki: Enabling real-time embodied ai robots via algorithm-architecture co-design. *arXiv preprint arXiv:2407.04292*, 2024.
- [29] Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *Advances in neural information processing systems*, 31, 2018.
- [30] Kemou Jiang, Xuan Cai, Zhiyong Cui, Aoyong Li, Yilong Ren, Haiyang Yu, Hao Yang, Daocheng Fu, Licheng Wen, and Pinlong Cai. Koma: Knowledge-driven multi-agent framework for autonomous driving with large language models. *arXiv preprint arXiv:2407.14239*, 2024.
- [31] Karthik Karur, Nitin Sharma, Chinmay Dharmatti, and Joshua E Siegel. A survey of path planning algorithms for mobile robots. *Vehicles*

- 3(3):448–468, 2021.
- [32] Yoonsung Kim, Changhun Oh, Jinwoo Hwang, Wonung Kim, Seongryong Oh, Yubin Lee, Hardik Sharma, Amir Yazdanbakhsh, and Jongse Park. Dacapo: Accelerating continuous learning in autonomous systems for video analytics. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14, 2024.
- [33] Po-Chen Ko, Jiayuan Mao, Yilun Du, Shao-Hua Sun, and Joshua B Tenenbaum. Learning to act from actionless videos through dense correspondences. *arXiv preprint arXiv:2310.08576*, 2023.
- [34] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- [35] Srivatsan Krishnan, Behzad Boroujerdian, William Fu, Aleksandra Faust, and Vijay Janapa Reddi. Air learning: a deep reinforcement learning gym for autonomous aerial robot visual navigation. *Machine Learning*, 110:2501–2540, 2021.
- [36] Srivatsan Krishnan, Thierry Tambe, Zishen Wan, and Vijay Janapa Reddi. Autosoc: Automating algorithm-soc co-design for aerial robots. *arXiv preprint arXiv:2109.05683*, 2021.
- [37] Srivatsan Krishnan, Zishen Wan, Kshitij Bhardwaj, Ninad Jadhav, Aleksandra Faust, and Vijay Janapa Reddi. Roofline model for uavs: A bottleneck analysis tool for onboard compute characterization of autonomous unmanned aerial vehicles. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 162–174. IEEE, 2022.
- [38] Srivatsan Krishnan, Zishen Wan, Kshitij Bhardwaj, Paul Whatmough, Aleksandra Faust, Sabrina Neuman, Gu-Yeon Wei, David Brooks, and Vijay Janapa Reddi. Automatic domain-specific soc design for autonomous unmanned aerial vehicles. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 300–317. IEEE, 2022.
- [39] Ashwin Sanjay Lele, Muya Chang, Samuel D Spetalnick, Brian Crafton, Shota Konno, Zishen Wan, Ashwin Bhat, Win-San Khwa, Yu-Der Chih, Meng-Fan Chang, et al. A heterogeneous rram in-memory and sram near-memory soc for fused frame and event-based target identification and tracking. *IEEE Journal of Solid-State Circuits*, 59(1):52–64, 2023.
- [40] Ziyun Li, Yu Chen, Luyao Gong, Lu Liu, Dennis Sylvester, David Blaauw, and Hun-Seok Kim. An 879gops 243mw 80fps vga fully visual cnn-slam processor for wide-range autonomous exploration. In *2019 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 134–136. IEEE, 2019.
- [41] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.
- [42] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems (NeurIPS)*, 36, 2024.
- [43] Kehui Liu, Zixin Tang, Dong Wang, Zhigang Wang, Bin Zhao, and Xuelong Li. Coherent: Collaboration of heterogeneous multi-robot system with large language models. *arXiv preprint arXiv:2409.15146*, 2024.
- [44] Qiang Liu, Zishen Wan, Bo Yu, Weizhuang Liu, Shaoshan Liu, and Arijit Raychowdhury. An energy-efficient and runtime-reconfigurable fpga-based accelerator for robotic localization systems. In *2022 IEEE Custom Integrated Circuits Conference (CICC)*, pages 01–02. IEEE, 2022.
- [45] Shaoshan Liu, Zishen Wan, Bo Yu, and Yu Wang. *Robotic computing on fpgas*. Springer, 2021.
- [46] Weizhuang Liu, Bo Yu, Yiming Gan, Qiang Liu, Jie Tang, Shaoshan Liu, and Yuhao Zhu. Archytas: A framework for synthesizing and dynamically optimizing accelerators for robotic localization. In *2021 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 479–493, 2021.
- [47] Yang Liu, Weixing Chen, Yongjie Bai, Jingzhou Luo, Xinshuai Song, Kaixuan Jiang, Zhida Li, Ganlong Zhao, Junyi Lin, Guanbin Li, et al. Aligning cyber space with physical world: A comprehensive survey on embodied ai. *arXiv preprint arXiv:2407.06886*, 2024.
- [48] Jiaying Lu, Bo Pan, Jieyi Chen, Yingchaojie Feng, Jingyuan Hu, Yuchen Peng, and Wei Chen. Agentlens: Visual analysis for agent behaviors in llm-based autonomous systems. *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [49] Zhao Mandi, Shreeya Jain, and Shuran Song. Roco: Dialectic multi-robot collaboration with large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 286–299. IEEE, 2024.
- [50] Víctor Mayoral-Vilches, Jason Jabbour, Yu-Shun Hsiao, Zishen Wan, Martiño Crespo-Álvarez, Matthew Stewart, Juan Manuel Reina-Muñoz, Prateek Nagras, Gaurav Vikhe, Mohammad Bakhshalipour, et al. Robotperf: An open-source, vendor-agnostic, benchmarking suite for evaluating robotics computing system performance. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8288–8297. IEEE, 2024.
- [51] Sean Murray, Will Floyd-Jones, Ying Qi, Daniel J Sorin, and George Dimitri Konidaris. Robot motion planning on a chip. In *Robotics: Science and Systems*, volume 6, 2016.
- [52] Sean Murray, William Floyd-Jones, Ying Qi, George Konidaris, and Daniel J Sorin. The microarchitecture of a real-time robot motion planning accelerator. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–12. IEEE, 2016.
- [53] Anjali Narayan-Chen, Prashant Jayannavar, and Julia Hockenmaier. Collaborative dialogue in minecraft. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5405–5415, 2019.
- [54] Sabrina M Neuman, Radhika Ghosal, Thomas Bourgeat, Brian Plancher, and Vijay Janapa Reddi. Roboshape: Using topology patterns to scalably and flexibly deploy accelerators across robots. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–13, 2023.
- [55] Sabrina M Neuman, Brian Plancher, Thomas Bourgeat, Thierry Tambe, Srinivas Devadas, and Vijay Janapa Reddi. Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 674–686, 2021.
- [56] Dima Nikiforov, Shengjun Chris Dong, Chengyi Lux Zhang, Seah Kim, Borivoje Nikolic, and Yakun Sophia Shao. Rosé: A hardware-software co-simulation infrastructure enabling pre-silicon full-stack robotics soc evaluation. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–15, 2023.
- [57] OpenAI. Gpt-4. <https://openai.com/research/gpt-4>, 2023. Accessed: 2024-10-06.
- [58] Daniele Palossi, Antonio Loquercio, Francesco Conti, Eric Flamand, Davide Scaramuzza, and Luca Benini. A 64-mw dnn-based visual navigation engine for autonomous nano-drones. *IEEE Internet of Things Journal*, 6(5):8357–8371, 2019.
- [59] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [60] Shivansh Patel, Saim Wani, Unnat Jain, Alexander G Schwing, Svetlana Lazebnik, Manolis Savva, and Angel X Chang. Interpretation of emergent communication in heterogeneous collaborative embodied agents. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15953–15963, 2021.
- [61] Liam Patterson, David Pigorovsky, Brian Dempsey, Nikita Lazarev, Aditya Shah, Clara Steinhoff, Ariana Bruno, Justin Hu, and Christina

- Delimitrou. Hivemind: a hardware-software system stack for serverless edge swarms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, pages 800–816, 2022.
- [62] Sven Mikael Persson and Inna Sharf. Sampling-based a^* algorithm for robot path-planning. *The International Journal of Robotics Research*, 33(13):1683–1708, 2014.
- [63] Xavier Puig, Tianmin Shu, Shuang Li, Zilin Wang, Yuan-Hong Liao, Joshua B Tenenbaum, Sanja Fidler, and Antonio Torralba. Watch-and-help: A challenge for social perception and human-ai collaboration. In *International Conference on Learning Representations (ICLR)*, 2021.
- [64] Jacob Sacks, Divya Mahajan, Richard C Lawson, Behnam Khaleghi, and Hadi Esmaeilzadeh. Robox: an end-to-end solution to accelerate autonomous control in robotics. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 479–490. IEEE, 2018.
- [65] Deval Shah, Ningfeng Yang, and Tor M Aamodt. Energy-efficient real-time motion planning. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–17, 2023.
- [66] Ying Su, Zhan Ling, Haochen Shi, Jiayang Cheng, Yauwai Yim, and Yangqiu Song. Actplan-1k: Benchmarking the procedural planning ability of visual language models in household activities. *arXiv preprint arXiv:2410.03907*, 2024.
- [67] Amr Suleiman, Zhengdong Zhang, Luca Carlone, Sertac Karaman, and Vivienne Sze. Navion: A 2-mw fully integrated real-time visual-inertial odometry accelerator for autonomous navigation of nano drones. *IEEE Journal of Solid-State Circuits*, 54(4):1106–1119, 2019.
- [68] MLC team. MLC-LLM. <https://github.com/mlc-ai/mlc-llm>, 2023.
- [69] Paul Von Behren. Nvml: implementing persistent memory applications. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST)*, 2015.
- [70] Zishen Wan, Nandhini Chandramoorthy, Karthik Swaminathan, Pin-Yu Chen, Kshitij Bhardwaj, Vijay Janapa Reddi, and Arijit Raychowdhury. Mulberry: Enabling bit-error robustness for energy-efficient multi-agent autonomous systems. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 746–762, 2024.
- [71] Zishen Wan, Yuhang Du, Mohamed Ibrahim, Yang Zhao, Tushar Krishna, and Arijit Raychowdhury. Thinking and moving: An efficient computing approach for integrated task and motion planning in co-operative embodied ai systems. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–7, 2024.
- [72] Zishen Wan, Yiming Gan, Bo Yu, Shaoshan Liu, Arijit Raychowdhury, and Yuhao Zhu. The vulnerability-adaptive protection paradigm. *Communications of the ACM*, 67(9):66–77, 2024.
- [73] Zishen Wan, Ashwin Lele, Bo Yu, Shaoshan Liu, Yu Wang, Vijay Janapa Reddi, Cong Hao, and Arijit Raychowdhury. Robotic computing on fpgas: Current progress, research challenges, and opportunities. In *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, pages 291–295. IEEE, 2022.
- [74] Shu Wang, Muzhi Han, Ziyuan Jiao, Zeyu Zhang, Ying Nian Wu, Song-Chun Zhu, and Hangxin Liu. Llm⁺ 3: Large language model-based task and motion planning with motion failure reasoning. *arXiv preprint arXiv:2403.11552*, 2024.
- [75] Yuanfei Wang, Fangwei Zhong, Jing Xu, and Yizhou Wang. Tom2c: Target-oriented multi-agent communication and cooperation with theory of mind. *arXiv preprint arXiv:2111.09189*, 2021.
- [76] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *arXiv preprint arXiv:2302.01560*, 2023.
- [77] Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, HaoWei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, et al. Jarvis-1: Open-world multi-task agents with memory-augmented multi-modal language models. *arXiv preprint arXiv:2311.05997*, 2023.
- [78] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [79] Yue Wu, So Yeon Min, Yonatan Bisk, Ruslan Salakhutdinov, Amos Azaria, Yuanzhi Li, Tom Mitchell, and Shrimai Prabhumoye. Plan, eliminate, and track—language models are good teachers for embodied agents. *arXiv preprint arXiv:2305.02412*, 2023.
- [80] Xilinx. Xilinx zynq-7000 soc zc706 evaluation kit. <https://www.xilinx.com/products/boards-and-kits/ek-z7-zc706-g.html>. Accessed: 2024-10-06.
- [81] Zhiyuan Xu, Kun Wu, Junjie Wen, Jinming Li, Ning Liu, Zhengping Che, and Jian Tang. A survey on robotics with foundation models: toward embodied ai. *arXiv preprint arXiv:2402.02385*, 2024.
- [82] Bo Yu, Wei Hu, Leimeng Xu, Jie Tang, Shaoshan Liu, and Yuhao Zhu. Building the computing system for autonomous micromobility vehicles: Design constraints and architectural optimizations. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1067–1081. IEEE, 2020.
- [83] Wenhao Yu, Jie Peng, Yueliang Ying, Sai Li, Jianmin Ji, and Yanyong Zhang. Mhrc: Closed-loop decentralized multi-heterogeneous robot collaboration with large language models. *arXiv preprint arXiv:2409.16030*, 2024.
- [84] Yangbin Yu, Qin Zhang, Junyou Li, Qiang Fu, and Deheng Ye. Affordable generative agents. *arXiv preprint arXiv:2402.02053*, 2024.
- [85] Bin Zhang, Hangyu Mao, Jingqing Ruan, Ying Wen, Yang Li, Shao Zhang, Zhiwei Xu, Dapeng Li, Ziyue Li, Rui Zhao, et al. Controlling large language model-based agents for large-scale decision-making: An actor-critic approach. *arXiv preprint arXiv:2311.13884*, 2023.
- [86] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B Tenenbaum, Tianmin Shu, and Chuang Gan. Building cooperative embodied agents modularly with large language models. *International Conference on Learning Representations (ICLR)*, 2024.
- [87] Hongxin Zhang, Zeyuan Wang, Qiushi Lyu, Zheyuan Zhang, Sunli Chen, Tianmin Shu, Yilun Du, and Chuang Gan. Combo: Compositional world models for embodied multi-agent cooperation. *arXiv preprint arXiv:2404.10775*, 2024.
- [88] Yang Zhang, Shixin Yang, Chenjia Bai, Fei Wu, Xiu Li, Xuelong Li, and Zhen Wang. Towards efficient llm grounding for embodied multi-agent collaboration. *arXiv preprint arXiv:2405.14314*, 2024.
- [89] Zirui Zhao, Wee Sun Lee, and David Hsu. Large language models as commonsense knowledge for large-scale task planning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [90] Yijie Zhou, Xianhui Cheng, Qiming Zhang, Lei Wang, Wenchao Ding, Xiangyang Xue, Chunbo Luo, and Jian Pu. Algpt: Multi-agent cooperative framework for open-vocabulary multi-modal auto-annotating in autonomous driving. *IEEE Transactions on Intelligent Vehicles*, 2024.